

AI Study Assistant

End-to-End Code Engineering & Step-by-Step Architecture Guide

1. User Login Flow (`/login`)

Complete step-by-step function execution trace when a user logs in.

Step 1: Form Submit Interceptor — `handleSubmit(e)`

File: frontend/app/(auth)/login/page.tsx

■ **THEORY & PURPOSE:** When the user clicks 'Sign In', HTML forms default to refreshing the page. `handleSubmit` prevents browser reload, triggers input validation, sets UI loading state, calls authentication service, updates global `AuthContext`, and redirects to Dashboard.

```
// Function Definition: Form submit event 'e' receive karta hai
const handleSubmit = async (e: React.FormEvent) => {
  // 1. Browser HTML default page refresh ko block karta hai
  e.preventDefault();

  // 2. Local input validation run karta hai (email & password length check)
  const errs = validate();
  if (Object.keys(errs).length) { setErrors(errs); return; }

  // 3. UI Loading State true karta hai (Login button spinner active)
  setLoading(true);
  try {
    // 4. authService.login() ko call karta hai form inputs { email, password } pass karke
    const { user } = await authService.login(form);
    // 5. AuthContext.login() call karke React global state update karta hai
    login(user);
    // 6. Next.js router se client-side view ko /dashboard par redirect kar deta hai
    router.push('/dashboard');
  } catch (err: any) {
    setErrors({ form: err.message });
  } finally {
    setLoading(false);
  }
};
```

Step 2: Input Validation Guard — `validate()`

File: frontend/app/(auth)/login/page.tsx

■ **THEORY & PURPOSE:** Client-side validation guard. Checks email format using *Regex* and password length before dispatching network request, saving server resources.

```
const validate = () => {
  const errs: Record<string, string> = {};
  // 1. Check if email field is empty
  if (!form.email) errs.email = 'Email is required.';
  // 2. Regex check: Ensures text has valid name@domain.ext format
  else if (!/\S+@\S+\.\S+/.test(form.email)) errs.email = 'Enter a valid email address.';

  // 3. Check if password field is empty or under 6 characters
  if (!form.password) errs.password = 'Password is required.';
  else if (form.password.length < 6) errs.password = 'Password must be at least 6 characters.';
  return errs;
};
```

Step 3: Frontend Data Packaging — authService.login()

File: frontend/lib/services/authService.js

■ **THEORY & PURPOSE:** Frontend Service Layer. Serializes credentials object to JSON, calls apiRequest(), stores returned JWT tokens in localStorage, and maps user data.

```
async login({ email, password }) {
  // 1. Sends HTTP POST request to NestJS /auth/signin endpoint
  const data = await apiRequest('/auth/signin', {
    method: 'POST',
    body: JSON.stringify({ email, password }),
  });
  // 2. Saves accessToken and refreshToken to browser localStorage
  storeTokens(data);
  // 3. Returns formatted user object and access token
  return { user: mapUser(data.user), token: data.accessToken };
}
```

Step 4: Network Transport Client — apiRequest()

File: frontend/lib/services/authService.js

■ **THEORY & PURPOSE:** Centralized HTTP Fetch wrapper. Sends HTTP POST to NestJS backend on port 3001, handles headers, parses JSON response, and handles error status codes.

```
async function apiRequest(endpoint, options = {}) {
  // 1. Executes native fetch call combining Base URL (http://localhost:3001) + Endpoint
  const res = await fetch(`${API_URL}${endpoint}`, {
    ...options,
    headers: { 'Content-Type': 'application/json', ...(options.headers || {}) },
  });
  let data = {};
  try { data = await res.json(); } catch (_) {}
  // 2. Handles non-2xx HTTP status codes
  if (!res.ok) {
    const message = (Array.isArray(data?.message) ? data.message[0] : data?.message) || 'Something went wrong.';
    throw new Error(message);
  }
  return data;
}
```

Step 5: Backend Controller Gateway — signIn()

File: backend/src/auth/auth.controller.ts

■ **THEORY & PURPOSE:** NestJS Controller Route Gateway. Listens for POST /auth/signin, validates payload against SignInDto class-validator rules, and delegates to AuthService.

```
@Post('signin')
@HttpCode(HttpStatus.OK)
signIn(@Body() dto: SignInDto) {
  // NestJS Pipe validates incoming JSON against SignInDto, then calls AuthService
  return this.authService.signIn(dto);
}
```

Step 6: Core Database Logic & Password Verification — signIn()

File: backend/src/auth/auth.service.ts

■ **THEORY & PURPOSE:** Mastermind Authentication Service. Queries MongoDB for user by email, compares password hash using Bcrypt, generates signed JWT tokens, and stores hashed refresh token in MongoDB.

```
async signIn(dto: SignInDto) {
  // 1. MongoDB Query: Searches 'users' collection for document matching email
  const user = await this.userModel.findOne({ email: dto.email });
  if (!user) throw new UnauthorizedException('Invalid email or password');

  // 2. Bcrypt Check: Compares plain input password against stored hash in MongoDB
  const isPasswordValid = await bcrypt.compare(dto.password, user.password);
  if (!isPasswordValid) throw new UnauthorizedException('Invalid email or password');

  // 3. Generates signed JWT Access Token (15m) and Refresh Token (7d)
  const tokens = await this.generateTokens(user._id.toString(), user.email);
  // 4. Hashes Refresh Token and updates user document in MongoDB
  await this.saveRefreshToken(user._id.toString(), tokens.refreshToken);

  return { message: 'Signed in successfully', user: this.sanitizeUser(user), ...tokens };
}
```

2. User Signup Flow (`/signup`)

Complete step-by-step user registration, Bcrypt hashing, and MongoDB insertion trace.

Step 1: Signup Form Interceptor — `handleSubmit(e)`

File: `frontend/app/(auth)/signup/page.tsx`

■ **THEORY & PURPOSE:** *Intercepts signup form submit, runs strict registration validation, dispatches `authService.signup()`, activates global auth state, and navigates to Dashboard.*

```
const handleSubmit = async (e: React.FormEvent) => {
  e.preventDefault();
  const errs = validate();
  if (Object.keys(errs).length) { setErrors(errs); return; }
  setLoading(true);
  try {
    const { user } = await authService.signup(form);
    login(user);
    router.push('/dashboard');
  } catch (err: any) {
    setErrors({ form: err.message });
  } finally {
    setLoading(false);
  }
};
```

Step 2: Strict Registration Validation — `validate()`

File: `frontend/app/(auth)/signup/page.tsx`

■ **THEORY & PURPOSE:** *Validates Full Name min length (2 chars), Email Regex pattern, Password min length (8 chars), and password matching equality (`password === confirmPassword`).*

```
const validate = () => {
  const e: Record<string, string> = {};
  if (!form.fullName.trim()) e.fullName = 'Full name is required.';
  else if (form.fullName.trim().length < 2) e.fullName = 'Name must be at least 2 characters.';
  if (!form.email) e.email = 'Email is required.';
  else if (!/\S+@\S+\.\S+/.test(form.email)) e.email = 'Enter a valid email address.';
  if (!form.password) e.password = 'Password is required.';
  else if (form.password.length < 8) e.password = 'Password must be at least 8 characters.';
  if (form.password !== form.confirmPassword) e.confirmPassword = 'Passwords do not match.';
  return e;
};
```

Step 3: Core User Registration & MongoDB Creation — `signUp()`

File: `backend/src/auth/auth.service.ts`

■ **THEORY & PURPOSE:** *Checks for duplicate email in MongoDB, encrypts password using Bcrypt (10 salt rounds), inserts new user document in MongoDB 'users' collection, generates JWT tokens, and updates database.*

```

async signUp(dto: SignUpDto) {
  // 1. Checks duplicate email in MongoDB
  const existingUser = await this.userModel.findOne({ email: dto.email });
  if (existingUser) throw new ConflictException('An account with this email already exists');

  // 2. Bcrypt Hashing with 10 Salt Rounds
  const hashedPassword = await bcrypt.hash(dto.password, 10);
  // 3. MongoDB User Creation
  const user = await this.userModel.create({ name: dto.name, email: dto.email, password: hashedPassword });
  // 4. Token Generation & Refresh Token Save
  const tokens = await this.generateTokens(user._id.toString(), user.email);
  await this.saveRefreshToken(user._id.toString(), tokens.refreshToken);
  return { message: 'Account created successfully', user: this.sanitizeUser(user), ...tokens };
}

```

3. Password Reset Engine & Gmail SMTP

Complete 2-part Password Reset & Email Delivery Architecture.

Step 1: Crypto Token Generation & Nodemailer Dispatch — forgotPassword()

File: backend/src/auth/auth.service.ts

■ **THEORY & PURPOSE:** Generates 32-byte crypto rawToken, hashes token with SHA-256 for MongoDB storage, sets 15-minute expiration, builds 1-click URL, and dispatches HTML email via Gmail SMTP.

```

async forgotPassword(dto: ForgotPasswordDto) {
  const user = await this.userModel.findOne({ email: dto.email });
  if (!user) return genericResponse;

  // 1. Node.js Crypto 32-bytes random rawToken generation
  const rawToken = crypto.randomBytes(32).toString('hex');
  // 2. SHA-256 Hashing for MongoDB storage
  const hashedToken = crypto.createHash('sha256').update(rawToken).digest('hex');
  user.resetPasswordToken = hashedToken;
  user.resetPasswordExpires = new Date(Date.now() + 15 * 60 * 1000); // 15 Mins Expiry
  await user.save();

  // 3. Gmail SMTP Nodemailer Dispatch
  const resetUrl = `http://localhost:3000/reset-password?token=${rawToken}`;
  const transporter = nodemailer.createTransport({ host: 'smtp.gmail.com', port: 587, auth: { user: 'ahmedebay0301@gmail.com', pass: '...' } });
  await transporter.sendMail({ from: 'StudyAI Assistant', to: dto.email, subject: 'Password Reset', html: emailHtml });
  return genericResponse;
}

```

Step 2: SHA-256 Token Verification & Password Reset — resetPassword()

File: backend/src/auth/auth.service.ts

■ **THEORY & PURPOSE:** Re-hashes incoming URL token with SHA-256, verifies match & expiry ($\$gt$ Date.now()), encrypts new password with Bcrypt, clears reset tokens, and invalidates refresh tokens across all devices.

```
async resetPassword(dto: ResetPasswordDto) {
  // 1. Re-hashes URL token with SHA-256
  const hashedToken = crypto.createHash('sha256').update(dto.token).digest('hex');
  // 2. Queries MongoDB matching token AND Expiry Time ($gt > Current Time)
  const user = await this.userModel.findOne({ resetPasswordToken: hashedToken, resetPasswordExpires: {
    $gt: new Date() } });
  if (!user) throw new BadRequestException('Reset token is invalid or has expired');

  // 3. Encrypts new password with Bcrypt and clears session tokens
  user.password = await bcrypt.hash(dto.newPassword, 10);
  user.resetPasswordToken = null;
  user.resetPasswordExpires = null;
  user.refreshToken = null; // Forces re-login across all active devices
  await user.save();
  return { message: 'Password has been reset successfully' };
}
```

4. Sign Out & Session Revocation

Guaranteed Client & Server Token Cleanup Architecture.

Step 1: Guaranteed Client & Server Cleanup — `authService.logout()`

File: `frontend/lib/services/authService.js`

■ **THEORY & PURPOSE:** Sends Bearer token POST request to `/auth/logout`, and guarantees `LocalStorage` cleanup (`accessToken` & `refreshToken` removal) inside `finally` block regardless of network connectivity.

```
async logout() {
  const accessToken = localStorage.getItem('accessToken');
  try {
    if (accessToken) {
      await apiRequest('/auth/logout', { method: 'POST', headers: { Authorization: `Bearer ${accessToken}` } });
    }
  } catch (_) {}
  finally { clearTokens(); } // Guaranteed LocalStorage Token Deletion!
  return true;
}
```

Step 2: MongoDB Refresh Token Invalidation — `logout()`

File: `backend/src/auth/auth.service.ts`

■ **THEORY & PURPOSE:** Updates user document in MongoDB setting `refreshToken = null`, permanently invalidating stolen or old refresh tokens from generating new access tokens.

```
@UseGuards(JwtAuthGuard)
@Post('logout')
async logout(@Req() req: Request) {
  const userId = (req as any).user.sub;
  await this.userModel.findByIdAndUpdate(userId, { refreshToken: null });
  return { message: 'Logged out successfully' };
}
```

5. Dashboard & Live Analytics

MongoDB Date Aggregations, Recharts, and Real-Time Activity Logger.

Step 1: MongoDB Date Aggregation & Analytics — `getDashboardStats()`

File: `backend/src/subjects/subjects.service.ts`

■ **THEORY & PURPOSE:** Calculates total subjects, monthly subject creation trends (`{ createdAt: { $gte: startOfMonth } }`), uploaded files count, and weekly upload trends (`{ createdAt: { $gte: startOfWeek } }`).

```

async getDashboardStats(userId: string) {
  const userObjectId = new Types.ObjectId(userId);
  const startOfMonth = new Date(now.getFullYear(), now.getMonth(), 1);
  const startOfWeek = new Date(now); startOfWeek.setDate(now.getDate() - 7);

  const totalSubjects = await this.subjectModel.countDocuments({ userId: userObjectId });
  let subjectsThisMonth = await this.subjectModel.countDocuments({ userId: userObjectId, createdAt: {
    $gte: startOfMonth } });
  const materialsCount = await this.materialModel.countDocuments({ userId: userObjectId });
  let filesThisWeek = await this.materialModel.countDocuments({ userId: userObjectId, createdAt: { $gt
    e: startOfWeek } });

  return { totalSubjects, subjectsThisMonth: subjectsThisMonth || totalSubjects, uploadedFiles: materi
    alsCount, filesThisWeek: filesThisWeek || materialsCount };
}

```

6. Subjects Management & Cascade Delete

Real-Time CRUD operations and Hard Disk physical file cleanup engine.

Step 1: Cascade Cleanup (Hard Disk & DB) — remove()

File: backend/src/subjects/subjects.service.ts

■ **THEORY & PURPOSE:** Deletes subject document from MongoDB, queries associated materials, permanently unlinks physical files from server hard disk (`fs.unlinkSync`), and deletes material records from MongoDB.

```

async remove(userId: string, id: string) {
  // 1. Delete Subject Document from MongoDB
  await this.subjectModel.findOneAndDelete({ _id: new Types.ObjectId(id), userId: new Types.ObjectId(u
    serId) });
  // 2. Query all associated materials
  const materials = await this.materialModel.find({ subjectId: new Types.ObjectId(id), userId: new Typ
    es.ObjectId(userId) });
  // 3. HARD DISK CLEANUP: Unlinks physical files from /uploads/ directory
  for (const mat of materials) {
    const filePath = join(process.cwd(), 'uploads', mat.fileUrl.replace('/uploads/', ''));
    if (existsSync(filePath)) unlinkSync(filePath); // Hard disk file deletion!
  }
  // 4. Delete material documents from MongoDB
  await this.materialModel.deleteMany({ subjectId: new Types.ObjectId(id), userId: new Types.ObjectId(
    userId) });
  return { message: 'Subject and associated materials deleted successfully' };
}

```

7. Drag & Drop File Upload Engine

Real-time Byte Stream Progress (XHR) and Multer Disk Storage.

Step 1: XHR Upload Progress & Express Multer Storage — `uploadFile()`

File: `frontend/lib/services/subjectService.js` & `backend/src/materials/materials.service.ts`

■ **THEORY & PURPOSE:** *Intercepts HTML5 drop event, uses native XMLHttpRequest with `xhr.upload.onprogress` to calculate loaded/total byte stream (0% to 100%), Express Multer saves file to `/uploads/`, and backend recalculates subject progress (+25% per file up to 100%).*

```
// Frontend XHR Progress Listener
xhr.upload.onprogress = (e) => {
  if (e.lengthComputable) {
    const percent = Math.round((e.loaded / e.total) * 100);
    onProgress(percent); // Real-time 0% to 100% upload progress bar!
  }
};

// Backend Express Multer & Progress Recalculation
async uploadFile(userId: string, subjectId: string, file: Express.Multer.File) {
  const newMaterial = new this.materialModel({ name: file.originalname, fileUrl: `/uploads/${file.filename}`, size: `${(file.size / (1024*1024)).toFixed(1)} MB` });
  await newMaterial.save();
  // Recalculates subject progress: +25% per uploaded material up to 100%
  subject.filesCount = (subject.filesCount || 0) + 1;
  subject.progress = Math.min(100, subject.filesCount * 25);
  await subject.save();
}
```

8. Document Preview, Download & Toolbar Filters

PDF `#page=N` jumping, Mammoth.js DOCX rendering, and toolbar view toggles.

Step 1: Document Preview & View Controls

File: `frontend/app/(dashboard)/dashboard/subjects/[id]/page.tsx`

■ **THEORY & PURPOSE:** *PDF previews load inside iframe with dynamic `#page=N` parameter jumping. DOCX files fetch binary ArrayBuffer and convert to formatted HTML via Mammoth.js. Toolbar pills filter array by extension (`.pdf` vs `.docx`) and switch layout between CSS 2-column Grid and compact vertical List.*

```
// 1. PDF Dynamic Page Jump Iframe
<iframe src={`http://localhost:3001${previewFile.url}#page=${previewPage}`} className='w-full h-[650px]' />

// 2. Mammoth.js DOCX to HTML Converter
const buffer = await res.arrayBuffer();
const result = await mammoth.convertToHtml({ arrayBuffer: buffer });
setDocxHtml(result.value);

// 3. Computed Extension Filtering & View Toggle
const filteredMaterials = materials.filter(m => {
  if (filterType === 'pdf') return m.name.endsWith('.pdf');
  if (filterType === 'docx') return m.name.endsWith('.docx');
  return true;
});
```

9. Settings Page (Profile vs Security)

Profile update with React AuthContext sync and Bcrypt password verification.

Step 1: Profile Update & Password Bcrypt Change

File: frontend/app/(dashboard)/dashboard/settings/page.tsx & backend/src/auth/auth.service.ts

■ **THEORY & PURPOSE:** Profile update verifies email availability in MongoDB, saves updates, and syncs React AuthContext so Navbar/Sidebar update live without reload. Password security change verifies current password via bcrypt.compare and hashes new password with bcrypt.hash(10).

```
// Profile Update (PUT /auth/profile)
const res = await authService.updateProfile({ name, email });
login(res.user); // Instantly updates Navbar & Sidebar user name!

// Password Security Change (POST /auth/change-password)
const isMatch = await bcrypt.compare(dto.currentPassword, user.password);
if (!isMatch) throw new BadRequestException('Current password is incorrect');
user.password = await bcrypt.hash(dto.newPassword, 10);
await user.save();
```